ViewFinder Initial Architecture Document

Team Number: Group 16

Team Members: Aayush Gajakas, Aiman Boullaouz, Alice Mungamuri, Audrey Facer, Heidi Schieber, Ibaad Khatib, Lilly Tran, Zain Cheema

Project Name: ViewFinder

Project Synopsis: ViewFinder is a community web platform where users discover, share, and explore scenic viewpoints through an interactive map and geotagged photos.

Architecture Description

Overview

ViewFinder is a three-tier web application made up of a browser-based single-page application (SPA), a stateless REST API, and two data stores: a relational database for structured data and object storage for photos. Sprint 1 builds the thinnest working slice through all three tiers, covering requirements 1–8 (40 story points), so that every later sprint adds features to an already deployed system rather than integrating late. Three rules govern the design. The browser talks only to the API, and the API alone touches the database and photo storage. Reading viewpoints is public, while every request that changes data must carry a valid token. Secrets are kept in environment variables, never in code.

Component View

The React (Vite) SPA is the only thing a visitor loads. It contains five UI modules: the Map View (requirements 2 and 5), the List View (requirement 4), the Detail View (requirement 6), the Auth Forms (requirement 7), and the Upload Form (requirement 8). They share one responsive stylesheet (requirement 3) and one API client. The Map View uses Leaflet and pulls base tiles from OpenStreetMap, an external service, so the team hosts no map data.

Every data request goes to the Node/Express REST API as JSON over HTTPS. The API has three route groups: Auth (register, log in, profile), Viewpoints (list, read, create), and Photos (store an image and return its URL). Each route calls a thin service layer so routes stay small and testable. A JWT-checking middleware guards every route that changes data and returns 401 Unauthorized when the token is missing or invalid. The service layer

reads and writes PostgreSQL for users, viewpoints, and photo records, and saves image files in object storage, keeping only each file's URL in the database. Figure 1 shows these components and the interfaces between them.

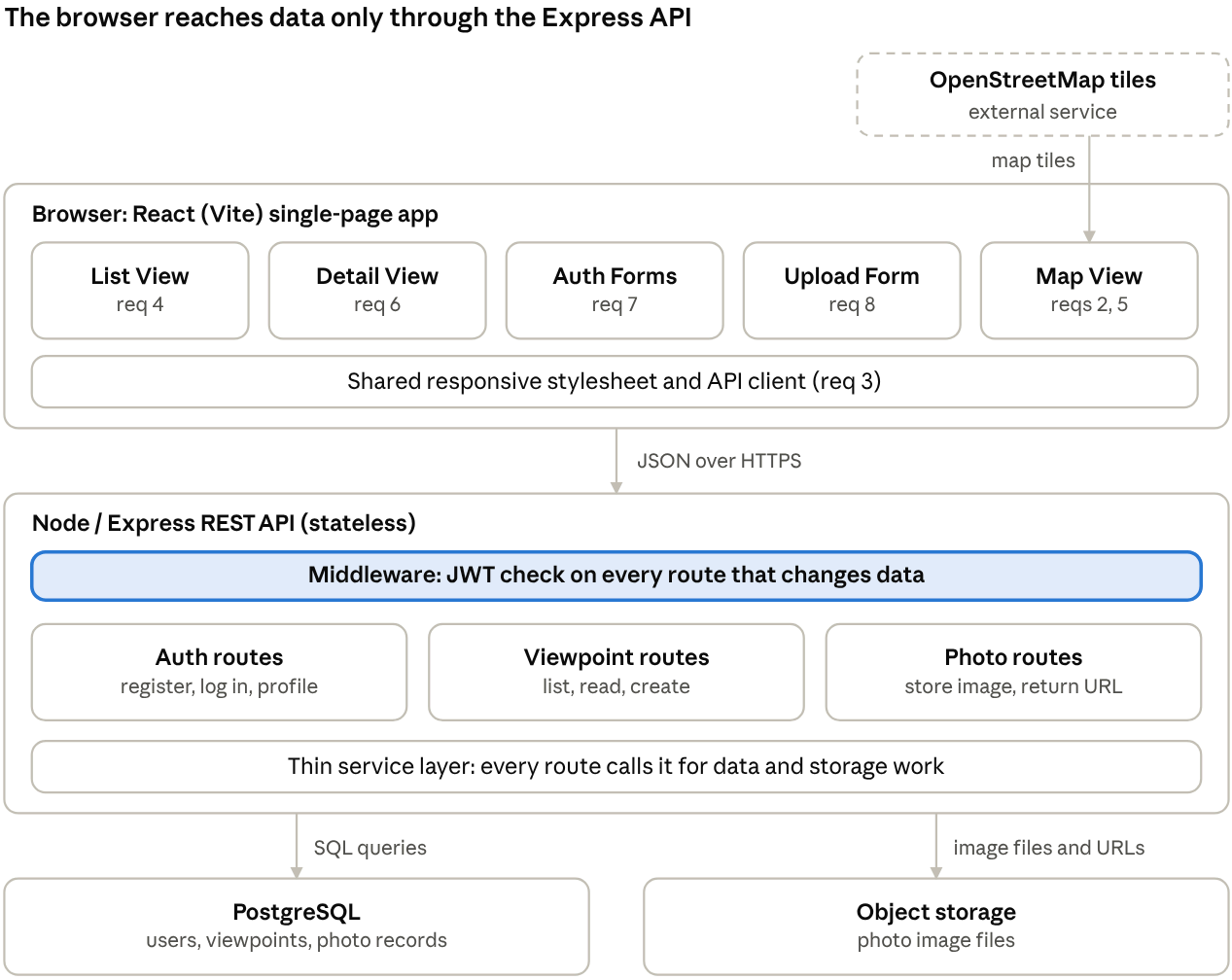


Figure 1 · UML component diagram · 3 tiers, 2 data stores

Figure 1. UML component diagram: the browser talks only to the API, and the API alone touches the database and photo storage.

Interaction View

Figure 2 shows the most important Sprint 1 interaction, a registered user logging in and uploading a viewpoint, because it touches every tier. In the login phase, the user submits an email and password, and the SPA sends POST /api/login. The API finds the user by email, compares the password with the stored salted hash, and returns a signed JSON Web Token (JWT), which the SPA keeps in memory and sends in the Authorization header from then on. In the upload phase, the user submits the form and the SPA posts the

viewpoint with the token. The API first verifies the token, then stores the photo in object storage, receives the photo URL, and inserts a Viewpoint row containing the title, description, coordinates, and photo URL. It responds with 201 Created, and the SPA adds a pin to the map and an entry to the list without reloading the page. Because verification happens before any write, an unauthenticated request can never create a file or a database row.

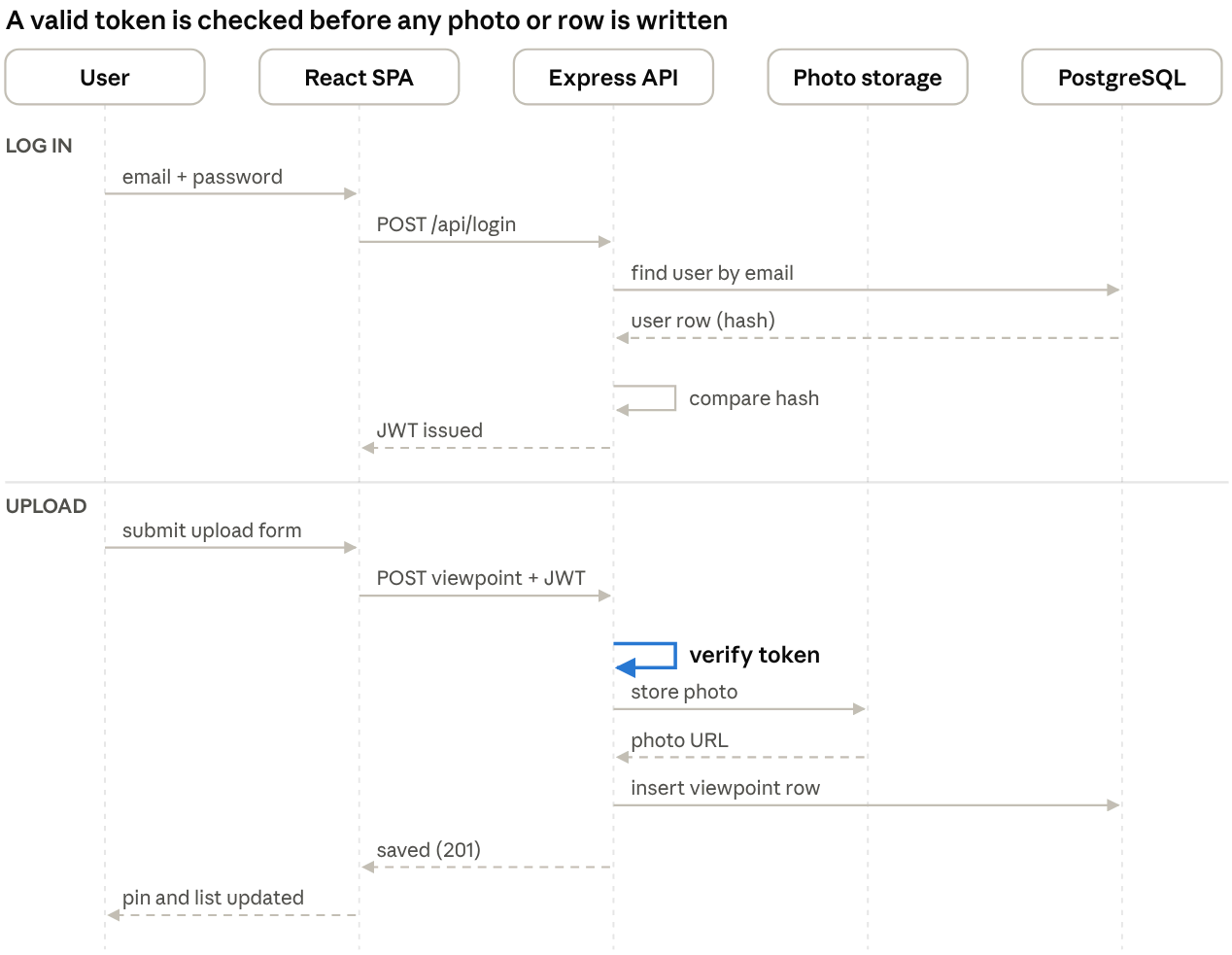


Figure 2 · UML sequence diagram · log in, then upload

Figure 2. UML sequence diagram: the API verifies the token before it writes the photo or the database row. Dashed arrows are replies.

Data View

Sprint 1 needs three classes, shown in Figure 3. A User has an id, a unique email, a display name, a password hash, and a created date. A Viewpoint belongs to exactly one User, its creator, and has a title, a rich-text description, decimal latitude and longitude, and a

created date. A Photo belongs to one Viewpoint and stores the image URL, a sort order, and an upload date. One User creates zero or more Viewpoints, and one Viewpoint has one or more Photos. Sprint 1 uses only one photo per viewpoint, but the one-to-many relationship means the later image gallery (requirement 18) needs no database change. Later sprints add Review, Favorite, Report, and Category classes linked to Viewpoint and User without altering these core fields.

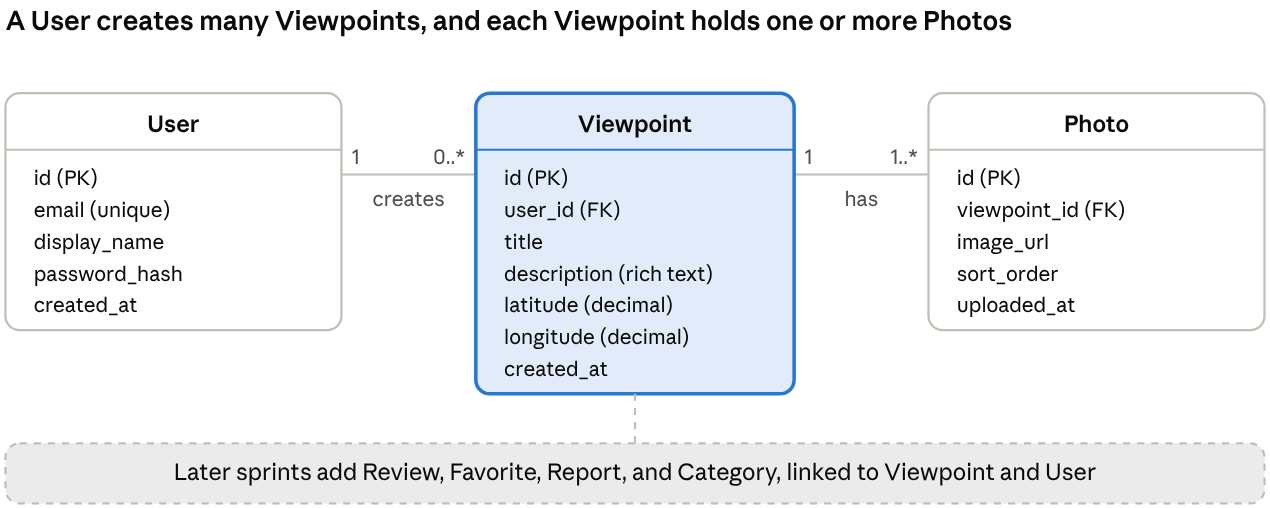


Figure 3 · UML class diagram · 3 classes, 2 associations

Figure 3. UML class diagram of the Sprint 1 data model. PK is a primary key and FK is a foreign key.

Deployment, Quality, and Extensibility

The SPA and API are deployed to a free-tier web host at a public HTTPS URL (requirement 1). The database and object storage are managed services reached through environment variables, so data survives a redeploy and no secrets live in the code. The layered design also supports the project's quality goals. Responsiveness (requirement 3) comes from mobile-first styles shared by all modules. Security (requirement 7) comes from salted password hashes, HTTPS-only authentication traffic, and token checks on every write. Maintainability comes from the thin service layer and from keeping the API stateless, which lets it scale horizontally later. Extensibility comes from the data model and route groups, which leave room for later requirements such as text search and category filters, Near Me radius search, reviews, favorites, and moderation without restructuring the system.